

Simple Routing Framework

This project uses a basic custom-built routing system to handle web requests. Here's a breakdown of how it works.

Core Components

1. `public/index.php` (The Entry Point)

- All public requests are directed to this file.
- It initializes the session, loads the Composer autoloader, and parses the incoming request URI and HTTP method.
- It includes the route definitions from `src/ConfigRoutes.php`.
- It dispatches the request to the router and uses a `Renderer` to display the final view.

2. `src/ConfigRoutes.php` (The Route Configuration)

- This file returns a function that defines all the application's routes.
- It's where you'll add new routes or modify existing ones.
- This keeps your routing configuration separate from the application's bootstrap logic.

3. `src/Router.php` (The Router)

- This class is responsible for storing all the defined routes.
- Its `dispatch()` method matches the current request's URI and method against the list of registered routes.
- If a match is found, it instantiates the corresponding controller and calls the specified method.
- If no route matches, it returns a 404 "Not Found" response.

4. Controllers (e.g., `src/Controller/HomeController.php`)

- Controllers contain the main application logic.
- Methods within a controller are linked to routes. When a route is matched, its associated controller method is executed.
- Controller methods are responsible for returning an array of "rendering options," which tells the `Renderer` which view to display and what data to pass to it.

How a Request is Handled

1. A request hits the server (e.g., `GET /exercises`).
2. The web server directs the request to `public/index.php`.
3. `index.php` creates a `Router` instance and then loads the route definitions from `src/ConfigRoutes.php`.
4. The script calls `$router->dispatch('/exercises', 'GET')`.
5. The `Router` finds the matching route: `['path' => '/exercises', 'method' => 'GET', 'handler' => [ExerciseController::class, 'index']]`.
6. It creates a new `ExerciseController` object and calls its `index()` method.
7. The `ExerciseController::index()` method runs and returns an array like `['view' => 'view/exercises/index', 'data' => [...]]`.

8. This array is passed to the **Renderer**, which includes the **gabarit.php** layout and injects the **view/exercises/index.php** view into it, along with its data.
9. The final HTML is sent to the browser.

How to Add a New Route

To add a new page (e.g., a "contact" page), you would:

1. **Create a new controller method** (or a new controller class if needed).

```
// In a controller like App\Controller\HomeController
public function contact(): array
{
    return ['view' => 'view/contact', 'data' => ['title' => 'Contact Us']];
}
```

2. **Create the corresponding view file** at **src/view/contact.php**.
3. **Register the new route** in **src/ConfigRoutes.php**.

```
$router->add('/contact', 'GET', [HomeController::class, 'contact']); // Add
this line
```